INTRODUCTION

E-commerce has revolutionized the retail industry by enabling large-scale digital transactions and generating extensive data across customer behaviour, purchasing patterns, payments, and logistics. This data provides valuable insights for analysing trends, optimizing operations, and enhancing customer experience.

This project utilizes the Brazilian E-Commerce Public Dataset provided by Olist — Brazil's largest department store marketplace. The dataset encompasses real commercial data from 2016 to 2018, containing information across multiple dimensions including orders, payments, products, customers, and seller interactions.

Objective: The primary objective of this project is to perform an end-to-end Exploratory Data Analysis (EDA) on the Olist dataset to uncover meaningful sales and revenue trends.

Tools- The analysis was performed using Python, leveraging libraries including Pandas for data manipulation, Matplotlib and Seaborn for visualization, and NumPy for numerical operations.

Dataset used: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

OBJECTIVE

Understanding sales and revenue trends is critical for any e-commerce business to make informed decisions around pricing, inventory, marketing, and logistics. Without structured analysis, patterns in revenue growth, payment behavior, and seasonal demand remain hidden within raw transactional data.

The analysis follows a structured pipeline: beginning with data loading and understanding, progressing through merging, cleaning, and transformation, and consequently leading to univariate, bivariate, and correlation analyses supported by meaningful visualizations.

The analysis is centered on **three** core datasets — **orders, order items, and payments** — focusing exclusively on delivered orders to ensure revenue integrity

IMPLEMENTATION STEPS

1. Mounting the drive and reading the files

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

orders = pd.read_csv('/content/drive/MyDrive/VAC
/olist_orders_dataset.csv')
items = pd.read_csv('/content/drive/MyDrive/VAC
/olist_order_items_dataset.csv')
payments = pd.read_csv('/content/drive/MyDrive/VAC
/olist_order_payments_dataset.csv')
```

2. Understanding the data

```
Orders.head()
Orders.info()
Orders.describe()
```

The file olist_orders_dataset.csv contained the following columns- order_id, customer_id, order_status, order_purchase_timestamp, order_apporoved_at,, order_delivered_carrier_date, order_delivered_customer_date and order_estimated_delivery_date with over 99440 entries.

In this, we have order_id as primary key to identify all the orders, order_status is used only to filter out “delivered” orders which we’ll further look upon in analysis. Order_purchased_timestamp is used to extract order_month, order_quater and day_of_week.

We have ignored the order_approved_at and order_delivered_carrier_date columns as they aren't relevant to revenue trends(which is our goal).

Items.head()
Items.info()
Items.describe()

The olist_order_items_dataset.csv contains the following columns- order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value with over 112650 records.

Here, we are using order_id as a foreign key which will be used to aggregate and merge with orders table. Order_item_id is used to count total items per order. price is is the actual product revenue, which is the most important column in the file because our analysis is centered around the revenue generation which is extracted from price. Freight_value is shipping cost per item, its used to calculate freight_ratio.
We have neglected product_id, seller_id and shipping_limit_data as they aren't required for analysing revenue trends.

Payments.head()
Payments.info()
Payments.describe()

The file payments.csv contains the features- order_id, payment_sequential, payment_type, payment_installments, payment_value with over 103885 entries.
Here, order_id is yet again, used as a forign key. Payment_type is important here as it tells "how" the transaction took place, it is used in bivariate analysis later on. Payment_installments will be correlated with revenue to understand purchase patterns, and lastly, payment_value is the actual amount paid.
In this file, we have ignored payment_sequential which isn't analytically useful.

3. Merging the dataset

```
items_agg = items.groupby('order_id').agg(
    total_items=('order_item_id', 'count'),
    revenue=('price', 'sum'),
    freight=('freight_value', 'sum')
).reset_index()

payments_agg = payments.groupby('order_id').agg(
    payment_value=('payment_value', 'sum'),
    installments=('payment_installments', 'max'),
    payment_type=('payment_type', lambda x: x.mode()[0])  #
most used type
).reset_index()


df = pd.merge(orders, items_agg, on='order_id', how='inner')
df = pd.merge(df, payments_agg, on='order_id', how='inner')
```

here, first I have aggregated items per order and then payments per order. In this scenario, aggregation before merging is important because, in these tables, one order_id has many rows in items table and similarly many rows in payments table. It's a **one-to-many** relationship which can cause a problem. If merged without aggregation, it can lead to "cartesian explosion".

4. Checking for null and duplicate values

```
df.isnull().sum()
df.duplicated().sum()
```

This block of code checks for total NULL values and duplicate values in the merged dataset.

5. Data cleaning

```
for col in ['order_purchase_timestamp',
'order_delivered_customer_date',
            'order_estimated_delivery_date']:
   df[col] = pd.to_datetime(df[col])

df = df[df['order_status'] == 'delivered']

df.dropna(subset=['revenue'], inplace=True)
```

Data cleaning is an essential step in data analysis, it makes the data accurate, consistent and usable.
In data cleaning we have performed 3 steps-

- Converting timestamp- when pandas read CSV, dates come in plain string format like-“2017-01-01 10:45:11”. Its difficult to use this format, so pd.datetime() converts them into proper datetime objects.
- Filtering- here we have filtered the order_status to only take “delivered” values. Because, revenue is only realized when a product is delivered. The other entries like cancelled, etc will be just noise in our analysis, and it can distort the numbers.
- Dropping null revenue rows- the entire analysis revolves around revenue, so, a row which doesn’t have a revenue value(null value) isn’t contributing to anything, hence not essential.

6. Data transformation

```
df['order_month'] =
df['order_purchase_timestamp'].dt.to_period('M')
df['order_year'] = df['order_purchase_timestamp'].dt.year
```

```
df['order_quarter'] =
df['order_purchase_timestamp'].dt.to_period('Q')
df['day_of_week'] =
df['order_purchase_timestamp'].dt.day_name()

df['avg_item_price'] = df['revenue'] / df['total_items']

df['freight_ratio'] = df['freight'] / (df['revenue'] + df['freight'])
```

Data transformation is also called feature engineering. It is essential to comprehend the meaning from raw data. Here also, we have performed 3 substeps:

- Time feature- here, from the order_purchase_timestamp , we have extracted order_month(M) and order_quarter(Q) and day_of_week.
- Average item price- the average_item_price helps us understand the price point customers are shopping at.
- Freight ratio- here, we have derived the freight ratio. So, its to understand- out of the total amount a customer pays for a product, what percentage of the amount goes towards shipping or logistics.

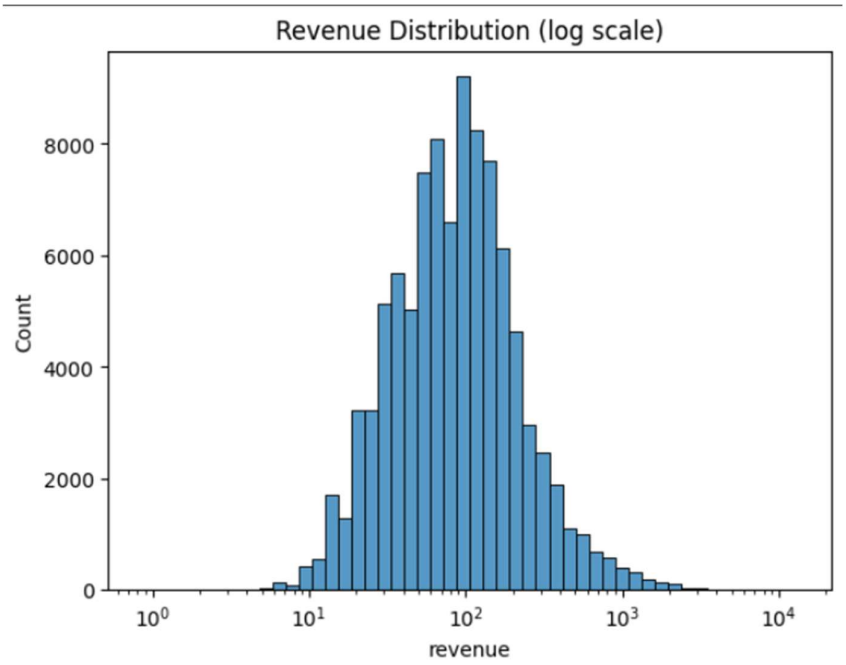
7. Univariate analysis

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df['revenue'], bins=50, log_scale=True)
plt.title('Revenue Distribution (log scale)')
plt.show()

df['payment_type'].value_counts().plot(kind='pie',
autopct='%1.1f%%')
plt.title('Payment Type Distribution')
plt.show()
```

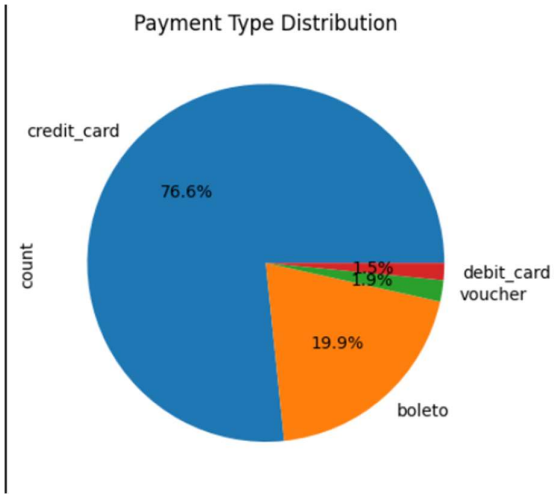
```
df.groupby('order_month')['order_id'].count().plot(figsize=(14,4)
)
plt.title('Monthly Order Volume')
plt.show()
```

in univariate analysis, as the name suggests “uni”- means we are examining or understanding each variable separately, or in isolation- its shape, range, outliers, and skew.

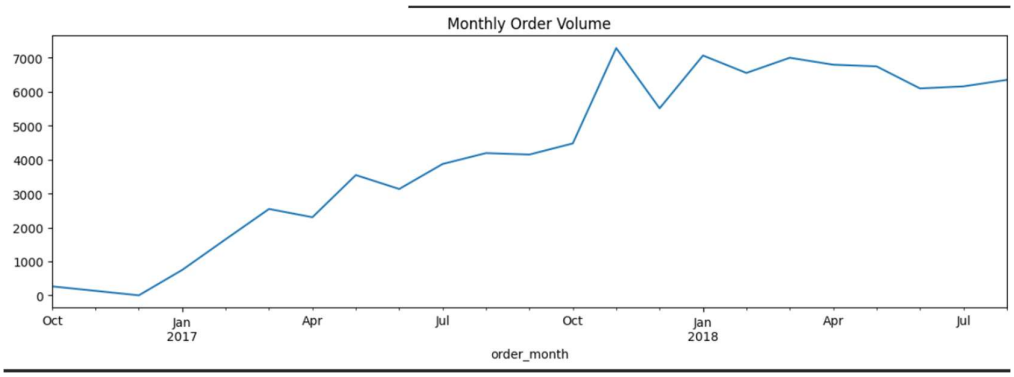
First we generated a plot on “revenue distribution”. In this, we used log scale because, by virtue, E-commerce revenue tends to be right skewed, as most orders are small(like $50-200) and few are large amounts (like $5000+). On a normal scale these outliers might compress the majority of the data , while log scale stretches the left and compresses the right , revealing the true shape.



Then we have plotted “payment type distribution”. This is done to check if one payment method is in clear majority or are the payment modes balanced.



Thirdly, we have plotted monthly order volume. This is an important baseline, as it checks if the number of orders are increasing monthly, as it'll affect the revenue.



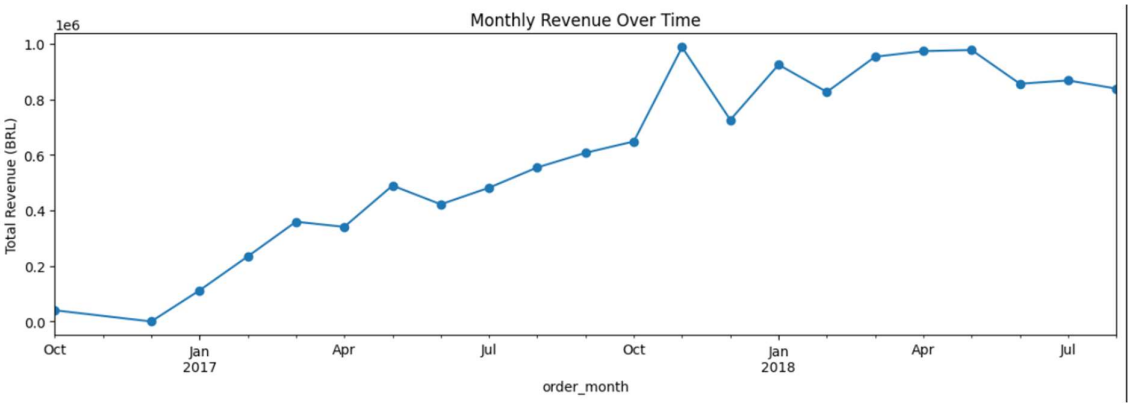
8. Bivariate analysis

```
monthly_rev = df.groupby('order_month')['revenue'].sum()
monthly_rev.plot(figsize=(14,4), marker='o')
plt.title('Monthly Revenue Over Time')
plt.ylabel('Total Revenue (BRL)')
plt.show()

df.groupby('payment_type')['revenue'].mean().sort_values().plot
(kind='barh')
plt.title('Avg Order Revenue by Payment Type')
plt.show()
```

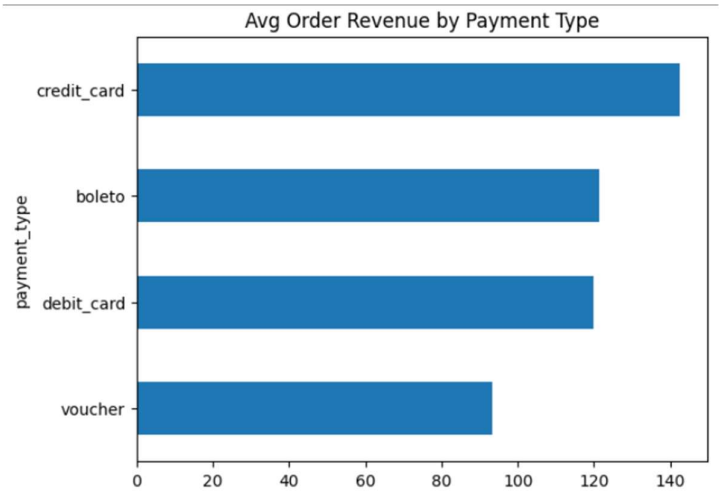
```
sns.boxplot(x='installments', y='revenue',
data=df[df['installments'] <= 12])
plt.title('Revenue vs Number of Installments')
plt.show()

df['day_of_week'].value_counts().reindex(
   ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday
','Sunday']
).plot(kind='bar')
plt.title('Orders by Day of Week')
plt.show()
```

Bivariate analysis involves finding relationship, patterns, and trends between two variables.

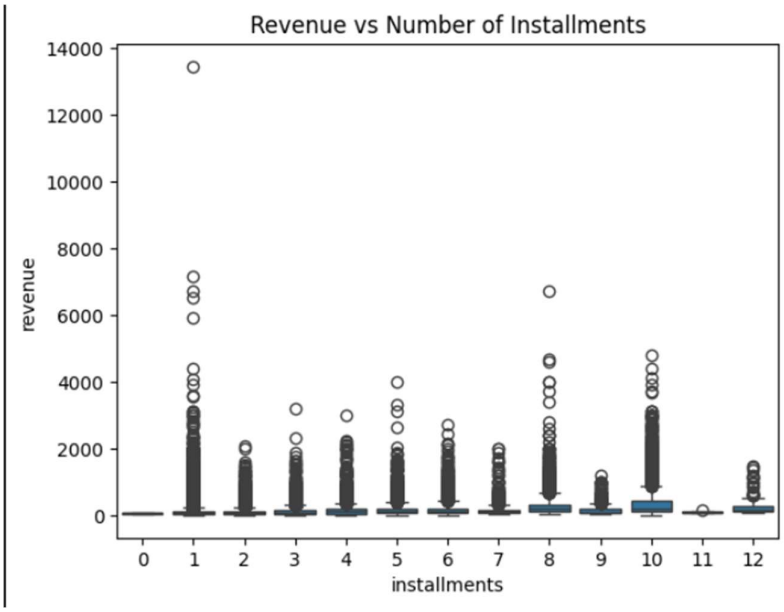
- Firstly we have “monthly revenue trend” , somewhat similar to what we did in monthly order volume in univariate analysis, it shows if the business is growing. We have used a line chart instead of bar graph, as it shows continuous change over time.



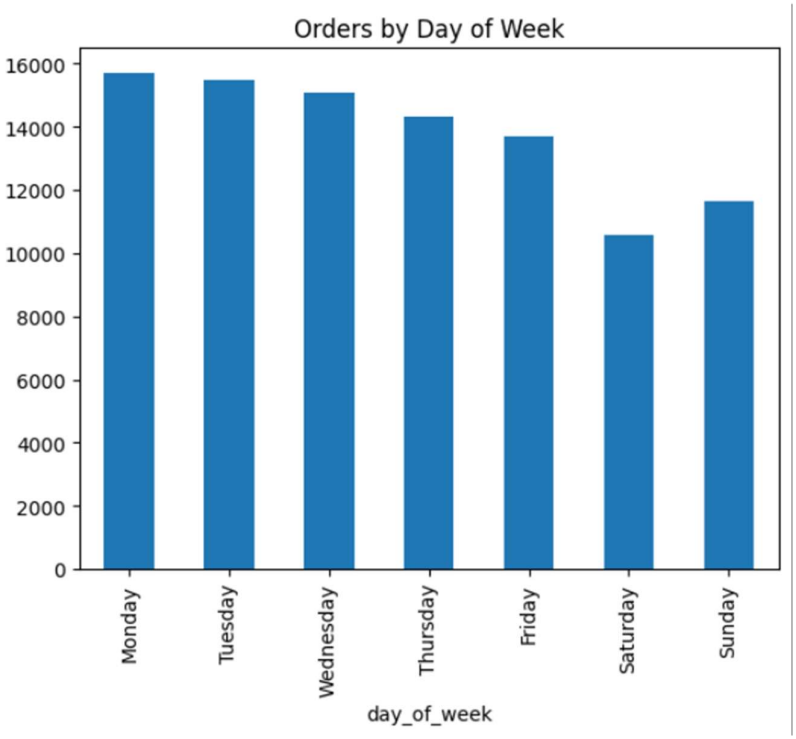
- The second plot is “revenue by payment type”. This analysis helps us understand relationship like- eg. “Do the customers who use credit cards tend to spend ore than those using boleto?”



- Third is a boxplot for instalment vs revenue- we have used a boxplot because it shows the distribution of revenue for each instalment count – median and outliers. It tests the hypothesis- “do people use more instalment for expensive purchases?”



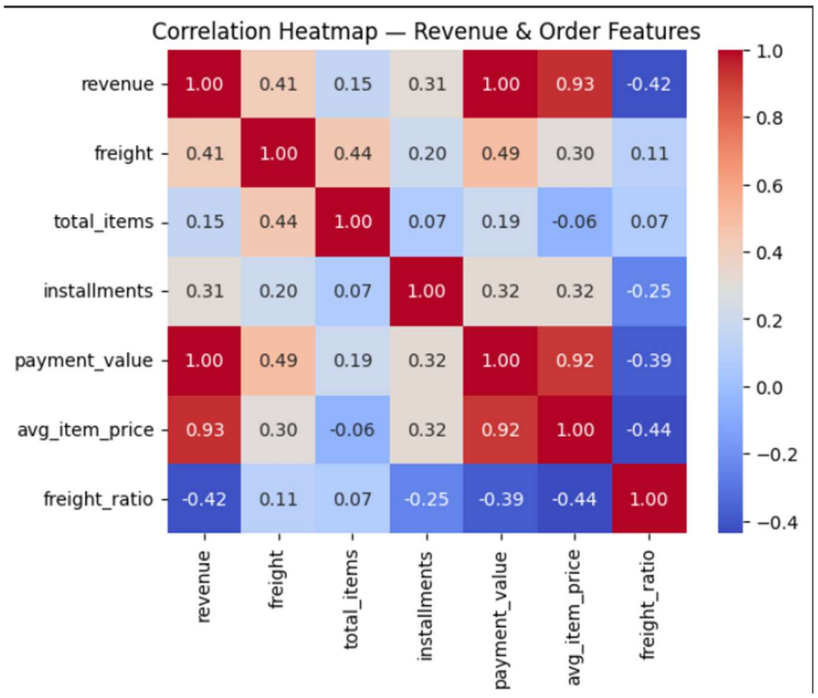
- Lastly in bivariate analysis is – day of week order. This gives insight on “when” the customers tend to do more purchases. This can help shape marketing strategies- when to organized campaigns, drop prices, flash sale, etc aiding in revenue growth.



9. Correlation analysis

```
corr_cols = df[['revenue', 'freight', 'total_items', 'installments',
               'payment_value', 'avg_item_price', 'freight_ratio']]

sns.heatmap(corr_cols.corr(), annot=True, fmt='.2f',
cmap='coolwarm')
plt.title('Correlation Heatmap — Revenue & Order Features')
plt.show()
```

So, bivariate analysis shows relationships visually, whereas, correlation analysis quantifies them with values between -1 and 1.
We used a heatmap here because, we have multiple numerical columns and heatmap helps us to see all pairwise correlations at ones.

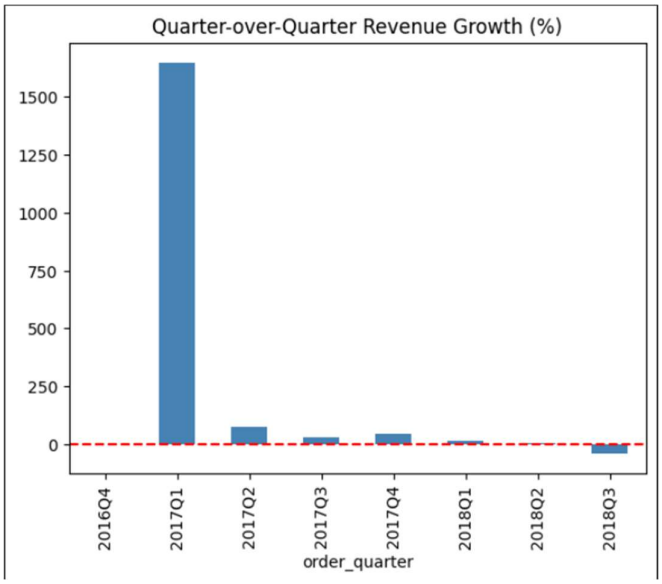


10. Visualization

```
qtr_rev = df.groupby('order_quarter')['revenue'].sum()
qtr_growth = qtr_rev.pct_change() * 100
qtr_growth.plot(kind='bar', color='steelblue')
plt.title('Quarter-over-Quarter Revenue Growth (%)')
plt.axhline(0, color='red', linestyle='--')
plt.show()

fig, ax1 = plt.subplots(figsize=(14,5))
monthly_rev.plot(ax=ax1, color='steelblue', label='Revenue')
ax2 = ax1.twinx()
df.groupby('order_month')['freight'].sum().plot(ax=ax2,
color='orange', label='Freight')
plt.title('Revenue vs Freight Cost Over Time')
fig.legend(loc='upper left')
plt.show()
```

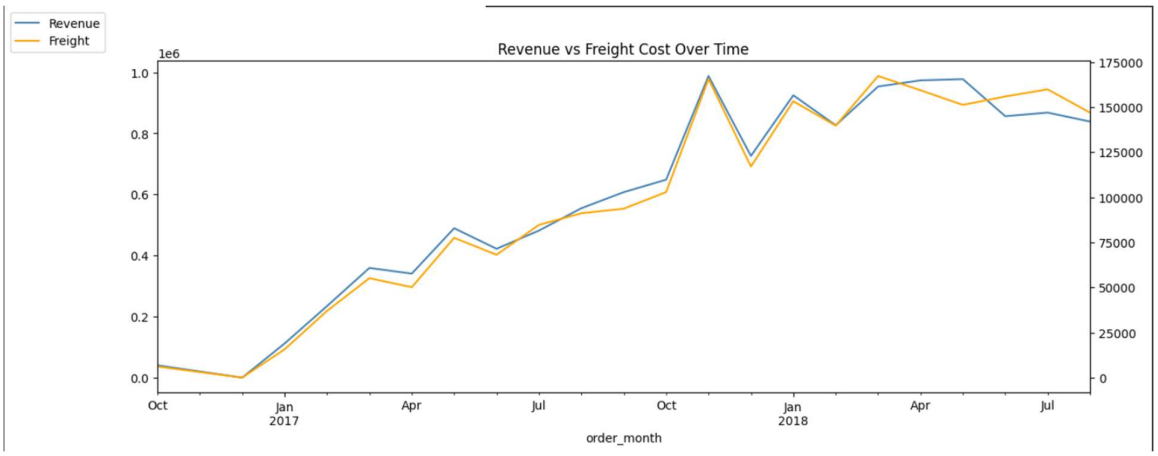
Visualization helps us create visuals which are self explanatory and allow us to communicate our point to stakeholders who may or may not be from a technical background.

- Quarter-over-quarter growth- we have used pct_change here, as a percentage values gives us more insights on how fast the business is growing, instead of large numbers.



- Revenue vs freight- we have used dual axis here as both revenue and freight have same units but very different scales. So, a twin axis allows both trends to be visble and comparable simultaneously

  Insight- “if freight cost grows faster than revenue over time, then it’s a problem, but if it grows proportionately, then its scaling well”



RESULT

The exploratory data analysis conducted on the Olist Brazilian E-Commerce dataset yielded several significant findings across revenue distribution, temporal trends, payment behaviour, and operational efficiency metrics.

- **Revenue Distribution and Order Value** Analysis of the revenue distribution revealed a strong right skew, indicating that the **majority of orders were concentrated in the lower to mid price range,** while a smaller proportion of high-value orders contributed disproportionately to total revenue.

  The average item price metric further highlighted that most customers engage in low-to-moderate value transactions, with premium purchases being relatively infrequent.
- **Temporal Trends** Monthly and quarterly revenue trends revealed a clear growth trajectory over the 2016 to 2018 period, with notable peaks suggesting the **influence of seasonal events and promotional campaigns on purchasing activity.**

  Quarter-over-quarter growth analysis showed periods of strong expansion interspersed with occasional contractions, reflecting the natural volatility of consumer demand.

  Order volume by day of week indicated that customers placed orders most frequently on weekdays, with activity tapering toward the weekend, suggesting that purchase decisions in this market are largely work-week driven.
- **Payment Behaviour Credit card emerged as the dominant payment method,** accounting for the largest share of transactions by a significant margin, followed by boleto bancário — a payment method unique to Brazil.

  A positive correlation between payment instalments and order revenue confirmed that **customers tend to break larger purchases into multiple instalments.**

  Orders paid via credit card also demonstrated higher average revenue compared to other payment types, indicating that credit card users represent a higher-value customer segment.
- **Freight and Operational Efficiency:** The freight ratio analysis revealed that **shipping costs constitute a meaningful proportion of total transaction value**, particularly for lower-value orders.

A negative correlation between order revenue and freight ratio confirmed that **larger orders are more freight-efficient** — the shipping cost is diluted across a higher product value. This suggests that encouraging higher cart values through bundling or minimum order incentives could improve both customer satisfaction and operational margins.

- **Correlation Analysis** The correlation heatmap validated several expected relationships — payment value and revenue showed near-perfect correlation, serving as a data integrity check.

  Total items per order showed a moderate positive correlation with revenue, confirming that cart size is a meaningful driver of order value. These relationships were statistically consistent and aligned with real-world business logic, lending confidence to the quality of the cleaned and merged dataset.

CONCLUSION

This project successfully demonstrated an end-to-end exploratory data analysis pipeline applied to a real-world e-commerce dataset.

Beginning with raw, transactional data spread across three separate tables, the analysis progressed through systematic merging, cleaning, transformation, and multi-level analysis(univariate, bivariate, correlational) to surface actionable business insights.

The findings collectively elucidates a clear picture of the Brazilian e-commerce landscape during 2016 to 2018 — a market in strong growth, dominated by credit card and instalment-based purchasing, with revenue concentrated among frequent low-to-mid value transactions and a freight cost structure that disproportionately burdens smaller orders. These insights have direct implications for business strategy, from payment infrastructure investment and promotional timing to logistics optimization and cart value maximization.

the project reinforced several foundational principles of data analysis. The importance of aggregating one-to-many relationships before merging, making deliberate business-driven cleaning decisions, engineering domain-relevant features, and selecting appropriate visualizations for each analytical question — these practices collectively determine the integrity and interpretability of any data analysis project.

It is important to note that these findings are descriptive rather than predictive.

Future work could extend this analysis into predictive modeling — forecasting revenue trends, predicting late deliveries, or segmenting customers by purchasing behavior — to move from insight to actionable decision support.

Colab notebook link- https://colab.research.google.com/drive/1eL2FrLuuYrZ5XoVxRR2muh8nZjrZ4c9x?usp=sharing